# xMCF
# ISO/PAS 8329

Description of mechanical connections and joints in structural systems

ISO/PAS 8329 χMCF WG meeting
focus group on issues #105 / #61
2026-04-01

CONTACT Software

Carsten.Franke@contact-software.com
https://www.linkedin.com/in/diplmathdrcarstenfranke/

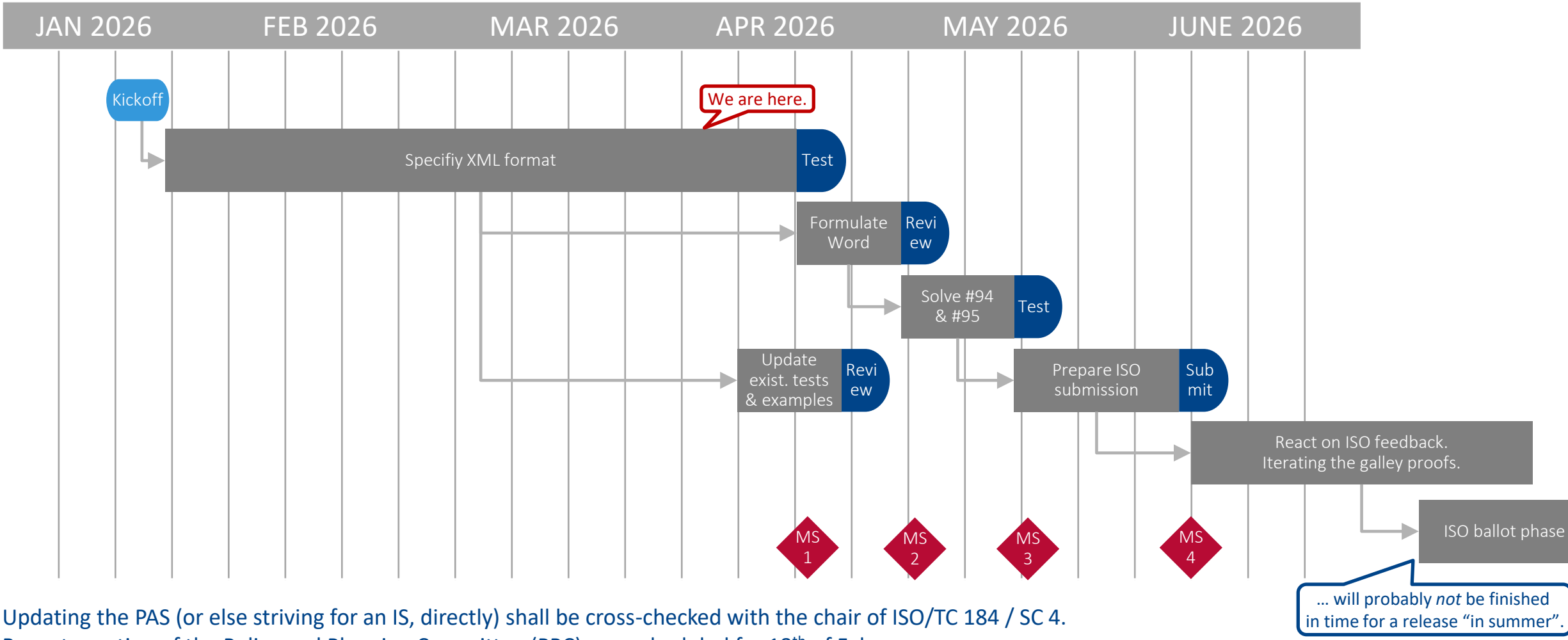
energizing great minds

# Agenda

As announced at https://github.com/economidis-nick/createXSDforxMCF/blob/master/VDA_FAT_AK_25/Meetings/2026-04-01_VideoConference/README.md

- Welcome & Flying over today's topics
- Timeline for next release
- How do we design templates in the XSD file? — Nick and Stephan's discussion at issue #105
- Focus on detailing the XML elements, based at issue #105
- Steps we should take next

# Timeline for the next release




Updating the PAS (or else striving for an IS, directly) shall be cross-checked with the chair of ISO/TC 184 / SC 4.
Recent meeting of the Policy and Planning Committee (PPC) was scheduled for 18th of Feb.

# Nick and Stephan's discussion at issue #105

as of 2026-03-25

Stephan summarized the approach as follows:

- Relax all relevant attributes from required to optional.
- Add "template" attributes inside the elements that were selected by the focus group.
- Add new elements for the templates themselves e.g. "washer_template".
- Use xs:assert to ensure that referenced templates exist.
- Use xs:assert to ensure that formerly required attributes are still present either in the template or at the position where the template is referenced.

However, some issues are left over. ⇒ Discussion:

- Nick's analysis is based on "paper documentation". Has not been verified by some example, yet.
- XML schema cannot verify a reference to an *external* template, contained in a file.
  Schematron would be able to do this.
  External template ∉ #105, but we should strive for a general solution which verifies both, internal and external templates.
- Next step: Verification of such an example by use of an XML tool. Joint effort by NE, GH & possibly KLT. Review by SH.
  Gum Drop & Rivet examples from different sides and then exchange.

# Result of Nick and Stephan's Investigation

See:

- Comments at issue #105, starting with comment of Mar. 29, 2026 and following.
- Commits
  57af94c — introduce <connection_templates> and <connection_0d> elements
  and
  0f1f398 — added schematron: first attempt.
- Files how to run schematron.txt,
  xmcf_3_2.sch,
  xmcf_3_2.xsd,
  and gumdrop_template.xml.

Commit comment

```
added schematron: first attempt
this schematron will assert
if   template/connection_0d contains a <spotweld>
and  connection_list/connection_0d contains a <gumdrop>

How to use it:

* Download Saxon-HE (free, Java-based).
* Download the ISO Schematron XSLT skeleton files
  from github (Schematron/stf)

** iso_dsdl_include.xsl
** iso_abstract_expand.xsl
** iso_svrl_for_xslt2.xsl
** iso_schematron_skeleton_for_saxon.xsl

Then, to test gumdrop_template.xml,
follow the steps in how_to_run_schematron.txt.
```

Remark: XML Schema V1.1 validation can be executed by our tool in GitHub at createXSDforxMCF/V3.1.1/test_suite.

# Conclusion of the Investigation

Combined XML verification by XML schema plus Schematron provides:

- Checks for several additional aspects, e.g. type confusion.
- Chance to separate business logic (↦ Schematron) and formal syntax (↦ Schema)

⇒ There are indisputable technical reasons to switch χMCF 3.2 validation to a Schematron-based method.

However:

- Implementing the complete Schematron file will take weeks.
- Many checks in XML schema need to be reduced from “mandatory” to “optional”. Will also take time.

NE & SH volunteer for both these items.

CF will care for updating the existing χMCF 3.2 example files.

# Detailing XML Elements — Template Syntax, Screw Example

We identified the example XML file chapter_9_5_7_exampleA__patched_for_xMCF3.2.xml (provided by the comment of 2026-10-14 09:17+0100) as a good starting point.

It suggests as a syntax for defining an (file-embedded) list of *connection templates* at XM[illegible]ot level:

```
<connection_templates>
    <connection_0d label="screw_template_4711">
        <threaded_connection part_code="M10x50 12.9" length="50." diameter="10"
            head_diameter="16.0" head_height="5" sink_size="4" thread_length="35" >
            <!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
            <screw /> <!-- Screw may come without any attributes -->
            <washer outer_diameter="20"/>
        </threaded_connection>
    </connection_0d>
</connection_templates>
```

If yes, the template must *not* have an attribute "label", since this would usually be *unique to each single* connector instantiated from the template.

For <washer/> and [illegible] we introduced the new attribute "[illegible] Shouldn't we use it here, too? (I[illegible]

Decision, 2026-03-11:
We change to „template_label" for all kind of templates.
➔ Draft example files to be updated.

Nick and Stephan's discussion at issue #105 may shed a new light on attribute naming.
→ See according slide!

Homework: Consider what "misuse" or contradictory set of data can occur by this approach!

# Detailing XML Elements — Reference Syntax, Screw Example

The example suggests as syntax for *referring to* a connection template at `<connection_0d/>` level:

Default form — no change to / overwriting template data:

```
<!-- Screw exactly as defined in connection_templates: -->
<connection_0d label="SCREW_100532" template="screw_template_4711">
    <threaded_connection>
        <normal_direction x="2.9" y="0.1" z="0.0" />
    </threaded_connection>
    <loc> 1500.3809 838.75885 730.6529 </loc>
</connection_0d>
```

Minimal change — overwriting an attribute value:

```
<!-- Screw exactly as defined in connection_templates, but with smaller torque: -->
<connection_0d label="SCREW_100533" template="screw_template_4711">
    <threaded_connection torque="70" >
        <normal_direction x="3.0" y="0.0" z="0.0" />
    </threaded_connection>
    <loc> 1540.3809 838.75885 730.6529 </loc>
</connection_0d>
```

# Detailing XML Elements — Reference Syntax, Screw Example

The example suggests as syntax for *referring to* a connection template at `<connection_0d/>` level:

Overwriting a child element:

```
<!-- Screw exactly as defined in connection_templates, but with larger washer: -->
<connection_0d label="SCREW_101537" template="screw_template_4711">
    <threaded_connection>
        <normal_direction x="3.1" y="-0.1" z="0.0" />
        <washer outer_diameter="30"/>
    </threaded_connection>
    <loc> 1580.3809 838.75885 730.6529 </loc>
</connection_0d>
```

Attention: Here, only the `outer_diameter` of the `<washer/>` is overwritten. Any other washer-parameter given from the template stays effective.

# Detailing XML Elements — Template Syntax, Bolt Example

The example extends to cases of *deeper structured* elements:

```
<connection_templates>
    <connection_0d label="bolt_template_4712">
        <threaded_connection part_code="M10x50 12.9" length="50" diameter="10"
            head_diameter="16.0" head_height="5" sink_size="4" thread_length="35" >
            <!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
            <normal_direction x="0" y="0" z="-10"/>
            <washer outer_diameter="20" inner_diameter="10.3" thickness="1.5" attached="true"/>
            <!-- Washer attached to the screw head -->
            <bolt>
                <nut diameter="16." height="5" />
                <!-- Friction attributes must not appear in a template,
                     unless it is against an attached washer -->
                <!-- Attributes "clipped_to" or "fixed_to" must not appear in a template,
                     since they each contain a flange partner index! -->
            </bolt>
        </threaded_connection>
    </connection_0d>
</connection_templates>
```

For `<washer/>` and `<nut/>`,
we introduced the new attribute "`template_label`".
Shouldn't we use it here, too? (Instead of "`label`"). (Cf. slide 7.)

Nick and Stephan's discussion at issue #105 may shed a new light on attribute naming.
→ See according slide!

# Detailing XML Elements — Reference Syntax, Bolt Example

The connection template can be *referred to* for example like this:

```
<!-- Bolt exactly as defined in connection_templates, but with less torque, larger washer & nut: -->
<connection_0d label="BOLT_100732" template="bolt_template_4712">
    <threaded_connection torque="70">
        <normal_direction x="3.1" y="-0.1" z="0.0"/>
        <washer outer_diameter="30"/>
Check, whether a washer may refer to a connection template, too!  → we need examples to better understand. → next slides!
        <bolt>
            <nut height="8"/>
Check, whether a nut may refer to a connection template, too! → we need examples to better understand. → next slides!
        </bolt>
    </threaded_connection>
</connection_0d>
```

Rule (→ Word file): If the connection template is a <screw/>, the referring element must not redefine it to a <bolt/> and vice versa!
To be extended to similar situations in case of future extensions of χMCF, e.g. a spotweld (from template) must not be redefined to a gumdrop!

# Detailing XML Elements — Template Syntax for a Washer

Draft example:

```
<!-- The new parameter "template_label" is allowed for a template, only. -->
<washer template_label="template for a loose washer"
    outer_diameter="20" inner_diameter="10.3" thickness="1.5"
    strength_property_class="12.9" part_code="M10x20x1.5 12.9" />
    <!-- Washer next to nut would have friction to last part, which is unknown. Thus, not allowed. -->
    <!-- A washer can be attached to a screw head or a nut.
         But here as a template without context, one can't know. Thus, "attached" is not allowed. -->
```

Remarks:

- Templates must not contain friction parameters, because friction always is a physical property of a *combination* of two items.
- We should differentiate between “firmly fixed” washers (i.e. washer and nut/screw form one solid)
  and “rotate-able” ones, where the washer and nut/screw can rotate independently around their common axis.
  Remark: BETA CAE’s study (2025-02-18) showed that we do *not* need a new attribute, here.

# Detailing XML Elements — Template Syntax for a Nut w/ Washer

Draft example:

```
<!-- The new parameter "template_label" is allowed for a template, only. -->
<nut template_label="template for a nut with an attached washer"
    diameter="16." height="5" strength_property_class="12.9" part_code="M10x5 nut 12.9"
    static_friction="0.8" kinetic_friction="0.6" >
    <!-- Attributes "torque" and "angle" must not appear in a template! -->
    <!-- Attributes "clipped_to" or "fixed_to" must not appear in a template,
         since they each contain a flange partner index! -->
    <!-- The washer of this nut is attached to it in such a way that it can rotate.
         Therefore, the nut's friction against the washer is meaningful and allowed. -->

    <!-- If the washer is part of or firmly attached to the nut, it cannot have a part code! -->
    <washer outer_diameter="25" thickness="1.5" strength_property_class="12.9"
        attached="true" static_friction="0.7" kinetic_friction="0.5"/>
    <!-- The friction of the washer describes the contact to the connected part. -->
    <!-- This washer could reference a template, itself. -->
</nut>
```

Remark: Friction between nut and attached but rotate-able washer is allowed!

# Detailing XML Elements — Template Syntax for a Bolt, using Washer & Nut templates, itself.

Draft example:

For <washer/> and <nut/>, we introduced the new attribute "template_label". Shouldn't we use it here, too? (Instead of "label"). (Cf. slide 7.)

Nick and Stephan's discussion at issue #105 may shed a new light on attribute naming. → See according slide!

```
<!-- The next bolt [illegible]ce references washer and nut templates, itself: -->
<connection_0d label="bolt_template_4733">
    <threaded_connection length="50" diameter="10" head_diameter="16" head_height="5" thread_length="35"
                         part_code="M10x50 12.9">
        <!-- Attributes "torque", "angle" and "pretension" must not appear in a template! -->
        <normal_direction x="0" y="0.8" z="-1.2"/>
        <washer template="template for a loose washer"/>  <!-- The washer next to the bolt's head. -->
        <bolt>
            <nut template="template for a nut with an attached washer"/><!-- Nut has attached washer.-->
        </bolt>
    </threaded_connection>
</connection_0d>
```

# Detailing XML Elements — Template Syntax, Gumdrop Example

## (new — see file “chapter_10_5_exampleB__with_templates.xml”)

For a gumdrop, the example template would be:

```
<connection_templates>
    <connection_0d label="GumDrop template">
        <!-- Assumed Unit system with mass attribute with value="kg" -->
        <gumdrop diameter="5.0" mass="0.0033" material="Glue_Material" />
    </connection_0d>
</connection_templates>
```

For `<washer/>` and `<nut/>`,
we introduced the new attribute “`template_label`”.
Shouldn’t we use it here, too? (Instead of “`label`”). (Cf. slide 7.)

Nick and Stephan’s discussion at issue #105 may shed a new light on attribute naming.
→ See according slide!

# Detailing XML Elements — Reference Syntax, Gumdrop Sequence

## (new — see file “chapter_10_5_exampleB__with_templates.xml”)

Decision, 2026-02-25:
<sequence_connection_0d/>
will *not* be considered within the scope of our current work, just as has been decided for all the other
<connection_1d/> types, before.

For a gumdrop sequence, the example reference could be (as suggested in GitHub comment):

```
<!-- Sequence of gum drops exactly as defined in connection_templates: -->
<connection_1d label="DROP_LINE_11000" template="GumDrop template">
    <sequence_connection_0d spacing="30.0" margin="1.0" priority="spacing">
        <gumdrop/>
    </sequence_connection_0d>
    <loc_list>
        …
    </loc_list>
</connection_1d>
```

Observation (CF): A template for a 0d item as parameter of a 1d item appears inappropriate to me.

What about this approach:

```
<connection_1d label="DROP_LINE_11000" template="Later, here could be place for a sequence template.">
    <sequence_connection_0d spacing="30.0" margin="1.0" priority="spacing" template="GumDrop template">
```

# XML Example File provided by @ghirel / PDM-IF.

The example “SAMPLE-FASTENER_003--.--1In Work000-ACOMMON_ENG_01_xmcf3.2 GHi.xml” has shown us that:

- `<loc/>` must not appear inside `<connection_templates/>`.
- `<stacking/>` must not appear inside `<connection_templates/>`.

# Contentual Action Items ≙ Next Steps:

- **Standard document (Word file):**
  - Description of overall concept & objectives, intended application & motivation etc.
  - Description of semantics for internal catalog
  - Description of semantics for reference to a connection template
  - Definition of example use cases and in-document XML-examples

- **XML & XSD** — See discussion in #105!
  - Sketching XML example file(s), incl. (w.i.p.)
    - precedence rule (w.i.p.) and dropped mandatory attribute,
    - internal (w.i.p.) and external file-based catalog
  - Description of syntax for internal catalogs (w.i.p.)
  - Description of syntax for reference to a connection template:
    - internal (w.i.p.), external file-based, external database-based
  - Drafting XML schema (XSD) and test cases / test example files
    Homework: Collect use cases which can yield e.g. test cases, shared as Markdown files in GitHub at https://github.com/economidis-nick/createXSDforxMCF/tree/issues_61_94_95_105/VDA_FAT_AK_25/Meetings/2026-02-18_VideoConference
  - Investigate χMCF elements and attributes for eligibility for template.
    ⇒ we need a rule, here! ⇒ Homework! (cf. Tables 35—79 of the standard)

# Next meetings

- 2026-04-08, 15:00 CEST — focus group on #105 / #61
  — objective: agree on XML suggestions & examples

- 2026-04-15, 15:00 CEST — t.b.d.
  — objective: t.b.d.

- 2026-04-22, 15:00 CEST — t.b.d.
  — objective: t.b.d.

CF sent / will send out invitations.